Lab 4

Locks, semaphores and one order

SENTINEL Lab 4: libsync. The last layer. Your four libraries make one centre.

Where Lab 4 fits

Lab 3 showed the fault: with no lock, the workers lost updates, and the stop did not end. Lab 4 removes the fault, and completes SENTINEL.

Lab 1 · Oct. 9

libsyscall

The interface to the operating system: the feed, the mission log and precise timestamps.

R6

Read a feed and lose none of it. Put a real date on each report. Record each report, in order.

Lab 2 · Oct. 30

libproc

One process for each sensor, and the channels that bring their reports to the centre.

R1 R2

Continue when one sensor stops. Let an urgent report overtake routine traffic.

Lab 3 · Nov. 20

libipc

Worker threads that share one processor and keep the track picture.

R3

Take thousands of reports each second. Measure what shared data with no lock costs.

Lab 4 · Dec. 4 · this lab

libsync

Protection for each thing that the workers share.

R4 R5

Keep the picture exact while many workers change it. Never freeze.

Lab 4 at a glance

Dec. 2

Prelab due by midnight. Necessary, no marks.

Dec. 4

Lab session. The Lab 3 oral exam is in this session.

Dec. 11

Deliverable due on Brightspace. 80 marks, 4% of the grade.

Dec. 11

Video due, with the zip. 20 marks, 4% of the grade. No oral exam.

The 80 marks of the deliverable

20	24	12	12	4	8
Acceptance test	Course tests	Your test	Code rules and errors	valgrind	The file agrees

What you write in Lab 4

`src/libsync/src/safetrack.c`	safe_track_init, safe_track_destroy, safe_track_update
`src/libsync/src/sem_queue.c`	sem_queue_init, sem_queue_destroy, sem_queue_push, sem_queue_pop
`src/libsync/src/resources.c`	resources_acquire_both, resources_release_both
`src/libsync/tests/test_sync.c`	Three test cases, with assert()

Read these first. Do not change them

`sync.h`

The steps and the errors of each function, and the calls that need no check.

`src/libsync/given/`

The worker pool, and the rest of the picture, the queue and the resources.

`src/sentinel/`

The final program. It uses your four libraries.

Your sem_queue.c calls your queue_push and queue_pop of Lab 3. A fault there is a fault in Lab 4.

Each fault of Lab 3, and its fix

The fault of Lab 3	Your code of Lab 4	What you see
Lost or repeated updates of the track picture	safe_track_update: one mutex around track_update	R4 shared data stays exact PASS
count of the queue below 0, and a stop that never ends	sem_queue_push and sem_queue_pop: two semaphores and a mutex	The ledger says BALANCED, and the run stops
A worker that polls an empty queue with usleep	sem_wait on full: the worker sleeps until a report comes	No processor time for an empty queue
Two workers that take two locks in opposite orders	resources_acquire_both: one fixed order	R5 never freezes PASS

The run of 8 workers on all processors that disagreed in Lab 3 agrees in Lab 4, in each run.

The ideas of Lab 4

A mutex protects the track picture

A mutex lets one worker at a time change the track picture; unlock it on each path, also after an error.

`pthread_mutex_lock()`

A semaphore puts an idle worker to sleep

A worker with an empty queue sleeps in sem_wait() and uses no processor time, until sem_post() tells it that a report arrived.

`sem_wait(), sem_post()`

Two semaphores make a bounded buffer

One semaphore counts the free slots and one counts the reports that wait; call sem_wait() before the lock, or a sleeping thread holds the lock.

`Lecture 3.3`

One lock order prevents a deadlock

Two workers that take two locks in opposite orders can wait for ever; take the lower rank first, and that circular wait cannot occur.

`Lecture 4.8`

Run it

Run each command at the top of your repository.

`make`	Builds src/libsync, then build/sentinel.
`make lab4`	Runs build/sentinel with your libraries for 3 s.
`make test`	Runs your test. With no lock, the case of two threads fails.
`make accept4`	Runs the 16 checks. A call that does not return ends after 30 s.
`make submit LAB=4`	Writes the zip of Lab 4.

Done when, then hand in

- [ ] make prints no warning, and make test passes.
- [ ] make accept4 gives 16 of 16.
- [ ] sentinel_8workers.txt says BALANCED, and PASS from R1 to R6.
- [ ] valgrind reports no error and no leak.
- [ ] make accept3 still gives 14 of 14.
- [ ] The video is 5 minutes or less, and follows the script.

Then hand in

```
make submit LAB=4
sentinel-lab4-<name>.zip
```

Put the file of the record run in docs/lab4/: sentinel_8workers.txt. It needs 2 processors or more.

Lab 4 Brief 6.1 gives the command.

Upload the zip and the video to Brightspace by Dec. 11.

4% The video, in place of the oral exam

1. The whole project

About 2 minutes · 8 marks

Run the centre, and show the ledger and R1 to R6. Say what SENTINEL does, which lab built each part, and what each requirement means.

2. What you solved in Lab 4

About 3 minutes · 12 marks

Show your mutex, your two semaphores and your lock order. For each one, say what goes wrong without it, and how your code prevents it.

At most 5 minutes, as an MP4 file: your screen and your voice. Use your own code and your own runs.

Questions?

If you have no questions, start work on Lab 4.

Read these two files first:

`docs/lab4/lab4_brief.pdf`

`src/libsync/include/sentinel/sync.h`